TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 10: Add Your Own System Call: trace

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 User interface and syscall number **5**

2 Kernel tracing **6**

2.1 Mask storage and dispatch 6

2.2 Read-only trace 6

3 Mask experiments **7**

Conclusions **8**

ABSTRACT

A new `trace` system call was added to xv6 on RISC-V. A bit mask selects calls for kernel-side logging, the setting is inherited by forked processes, and a user program applies the mask before replacing itself with the requested command. The observed checks cover syscall-number routing, filtered reads, and the handout's mask questions.

Keywords: xv6, RISC-V, system call, trace, syscall table, process, bit mask

INTRODUCTION

The lab adds a user-visible `trace mask command [args...]` utility to the xv6 RISC-V system. The implementation follows syscall number 23 through the user declaration and generated stub, kernel dispatch table, handler and per-process state. The handout's xv6 tree already assigns number 22 to `sync`, so the new call uses 23. The kernel uses `printk` for trace output.

The modified tree is the separate `xv6-riscv` working tree; the portable changes are included as `xv6-trace.patch`. The screenshots document the staged check: first the new number reaches the kernel but has no dispatch handler; after implementing the kernel side, tracing mask 32 logs the four reads from `grep`.

1 USER INTERFACE AND SYSCALL NUMBER

The user program validates its arguments, calls `trace(mask)`, then executes the requested command with `exec`. A successful `exec` preserves the process and its trace mask while replacing its program image.

The number is defined as `SYS_trace 23`. `user.h` declares `trace(int)`, and `usys.pl` generates the user stub. At build time, this stub places the number in the syscall register and executes `ecall`. The Makefile installs `trace` in the xv6 filesystem image.

The first screenshot shows both the usage check and the expected intermediate failure:

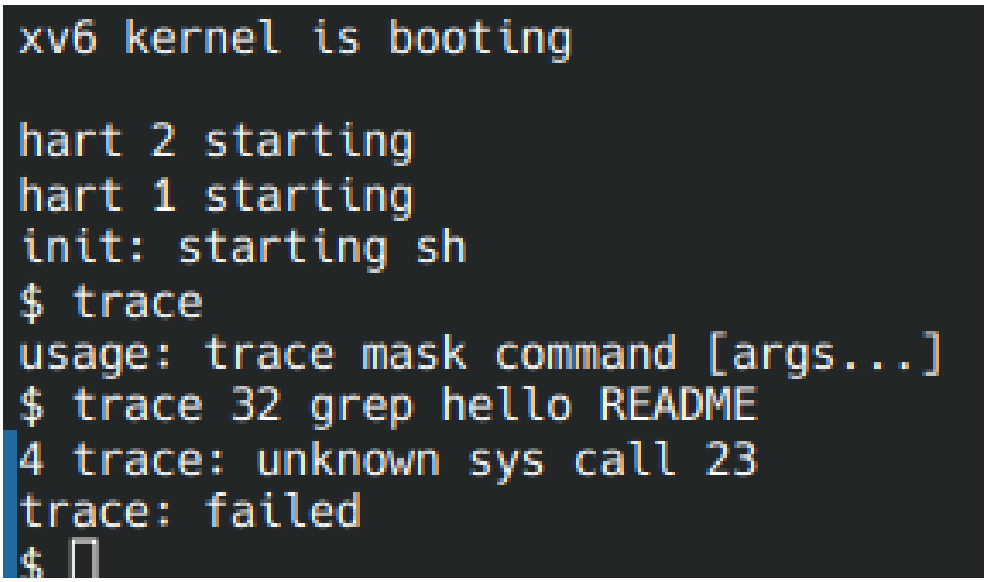


Figure 1.1 – User utility validation and syscall 23 before kernel dispatch was added

The message “unknown sys call 23” confirms that the user stub reached the kernel with the intended number. At this stage, the kernel has no handler registered for it.

2 KERNEL TRACING

2.1 Mask storage and dispatch

Each process stores its mask in `struct proc`. The `sys_trace` handler reads argument 0 and saves it for the current process. The syscall dispatch table maps number 23 to that handler. A syscall-name table maps syscall numbers to printable names.

After dispatch returns, the kernel checks bit `num` of the process mask. When set, it prints the PID, syscall name and return value. In `fork()`, the parent's mask is copied to the child with `np->trace_mask = p->trace_mask`; this allows traced commands that create children to remain traced.

2.2 Read-only trace

Mask 32 has bit 5 enabled, corresponding to `SYS_read`. Running `trace 32 grep hello README` produced:

```
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

The three positive results total 2441 bytes, the size of the xv6 `README`; the final zero marks end-of-file. The PID can vary between boots. The output is consistent with the screenshot and handout check.

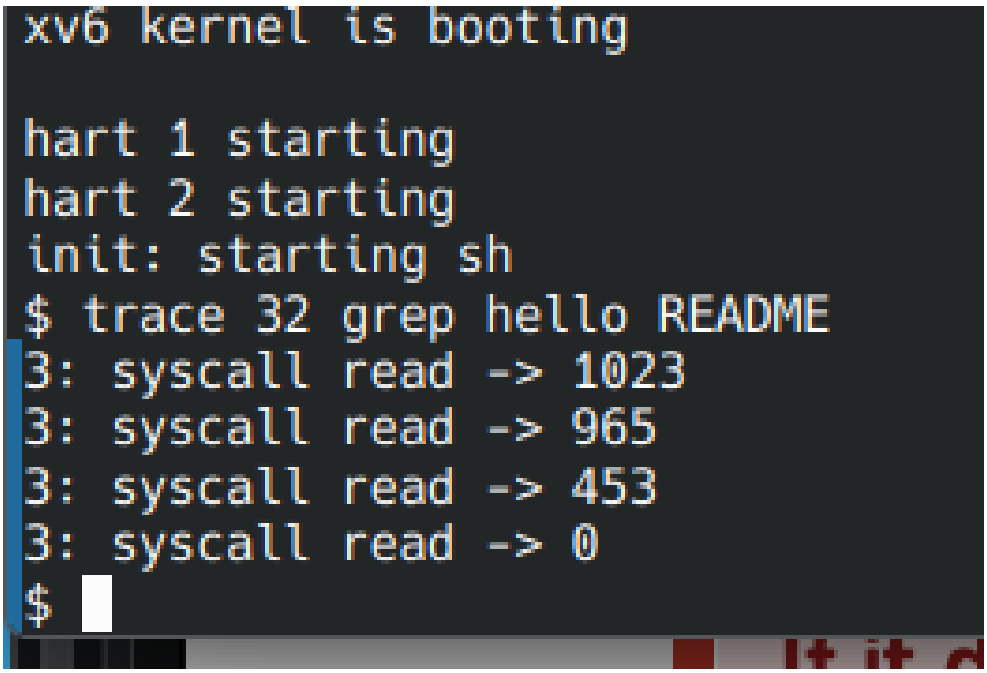


Figure 2.1 – The trace utility logging the selected read calls from grep

3 MASK EXPERIMENTS

1. With mask 32, only `read` is printed because bit 5 is set. Although `grep` opens and closes `README`, `open` is syscall 15 and `close` is syscall 21, whose bits are not selected.
2. In `trace_mask = p->trace_mask` inside `fork()`, each child receives its parent's mask. Therefore fork descendants also log selected calls.
3. `echo hi` writes twice: one write for `hi`, then another for the newline. The kernel prints the trace line after the first write returns, so it appears between the text and newline.
4. To select only `open` (15) and `close` (21), use `(1 << 15) | (1 << 21) = 2,129,920`.

CONCLUSIONS

The syscall path is complete from the user command to kernel dispatch and reporting. Per-process mask state allows selective logging, while copying it during `fork` extends tracing to child processes. The observed mask-32 run reports exactly the selected reads, including the zero-byte end-of-file result.